OSTAAD ERP

System Architecture and Functional Design

Pre-Construction, Project Management, BOQ, Procurement, Finance and Site Operations

Feature-Rich, Lightweight ERP

Single-Worker Cloudflare Full-Stack Architecture

Architecture Proposal

November 2026

Contents

1 Project Overview **2**
1.1 Core Objectives . . . 2
1.2 Architecture Philosophy . . . 2

2 High-Level System Architecture **2**

3 Technology Architecture **3**
3.1 Single Full-Stack Application . . . 3
3.2 Backend Runtime . . . 4

4 Single Root Project Structure **4**
4.1 Code Organisation Rules . . . 5

5 Cloudflare Infrastructure Design **5**
5.1 Cloudflare Worker . . . 5
5.2 Cloudflare D1 . . . 6
5.3 Cloudflare R2 . . . 6
5.4 Cloudflare KV . . . 6
5.5 Services Not Required Initially . . . 6

6 Functional Modules **7**
6.1 Module 1: Pre-Construction Planning . . . 7
6.1.1 Pre-Construction Workflow . . . 7
6.2 Module 2: Project Management . . . 8
6.3 Module 3: BOQ and Cost Management . . . 8
6.4 Module 4: Vendor Management and Procurement . . . 8
6.5 Module 5: Finance and Accounts . . . 9
6.6 Module 6: Site Operations and Inventory . . . 9
6.7 Module 7: Reports and Administration . . . 9

7 User Roles and Dashboards **9**

8 End-to-End Business Workflow **10**

9 Database Architecture **11**
9.1 Main Database Entities . . . 12
9.2 Database Design Rules . . . 12
9.3 Multi-Company Architecture . . . 12

10 API Architecture **12**

11 Documents and File Management **13**

12 Security, Reliability and Audit **13**

13 Deployment Architecture **14**
13.1 Deployment Responsibilities . . . 14

14 Development Phases **14**

15 Features Outside the Current Scope **15**

16 Final Architecture Summary **15**

1 Project Overview

Ostaad ERP is a construction business management platform designed to manage the construction project lifecycle, from pre-construction planning through procurement, finance, inventory and site operations.

The objective is to build an ERP that is rich in useful business features but lightweight in its technical design. The application should be straightforward to develop, deploy, operate and maintain.

The complete application will be hosted on Cloudflare. There will be one full-stack application, one deployed Cloudflare Worker, one central Cloudflare D1 database and Cloudflare R2 for file storage.

1.1 Core Objectives

- Manage all major pre-construction activities.
- Maintain project-wise estimates and budgets.
- Create, import, edit and manage BOQs.
- Compare vendors and manage quotations.
- Control procurement and purchase orders.
- Manage construction schedules and phases.
- Track permits and statutory approvals.
- Manage site inventory and material consumption.
- Track financial approvals, expenses and payments.
- Maintain project documentation and audit history.
- Support multiple companies through one application.

1.2 Architecture Philosophy

- Cloudflare-only production infrastructure.
- One integrated full-stack application.
- One Cloudflare Worker deployment.
- One root project directory and one codebase.
- Frontend pages and backend API routes in the same application.
- Cloudflare D1 as the only relational database.
- Cloudflare R2 for documents and file storage.
- No external PostgreSQL or third-party database.
- No separate frontend and backend deployments.
- No microservices at the initial stage.
- No unnecessary queues, services or infrastructure.
- Business functionality takes priority over technology complexity.

2 High-Level System Architecture

Ostaad ERP will use a single-application, modular monolithic architecture. The frontend and backend will be different logical parts of the same codebase and will be deployed together through one Cloudflare Worker.

The browser loads the application from the Worker. Frontend pages call API routes served by the same application. Those API routes execute the business logic and access D1 and R2 through Cloudflare bindings.

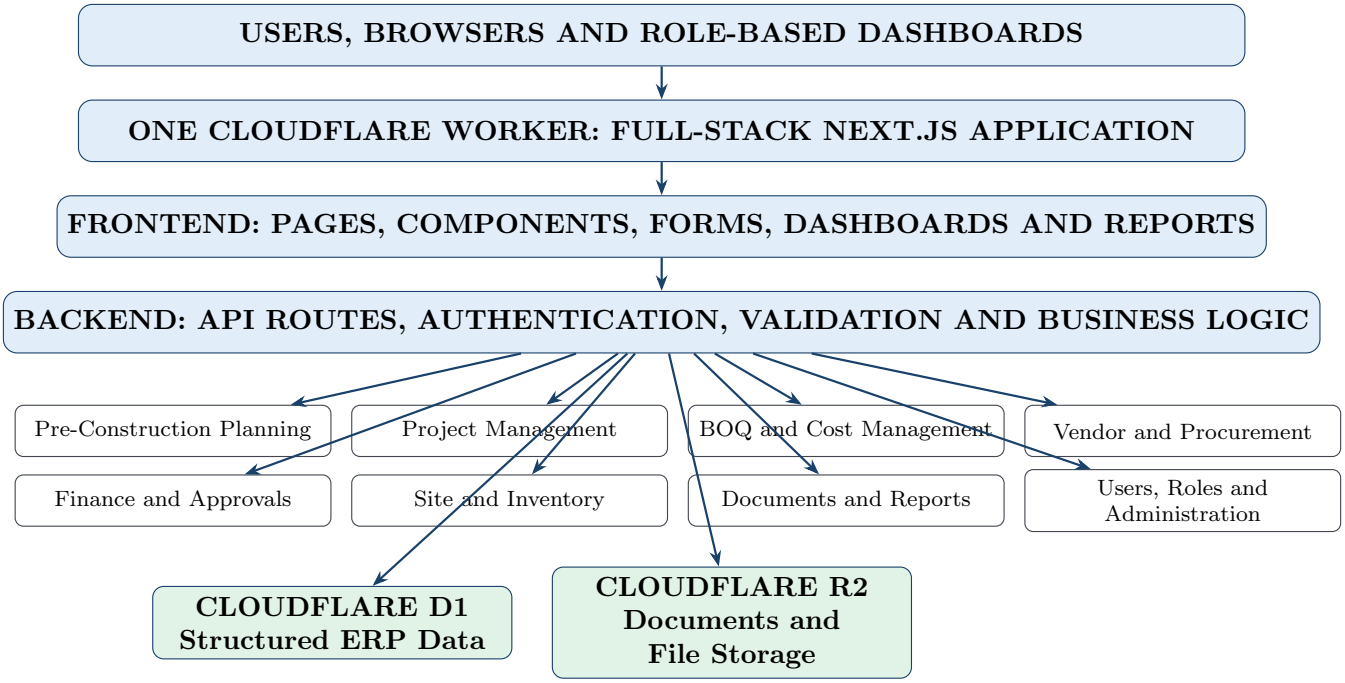


Figure 1: Single-Worker Cloudflare architecture for Ostaad ERP

Important: The boxes above represent logical layers and modules, not separate servers or deployments. The complete application is deployed as one Worker.

3 Technology Architecture

The application will use a lightweight full-stack architecture that fits the Cloudflare Workers runtime.

Component	Technology	Responsibility
Full-stack application	Next.js, TypeScript	Frontend pages, API routes, shared types and application logic.
Runtime and hosting	One Cloudflare Worker	Serves the application and executes server-side API logic.
Database	Cloudflare D1	Stores structured ERP records, relationships and transactions.
File storage	Cloudflare R2	Stores PDFs, drawings, quotations, invoices and attachments.
Optional cache	Cloudflare KV	Caches suitable non-critical lookup or configuration data only.
API communication	Same-origin REST/HTTP routes	Connects frontend interactions to backend API routes within the same application.
Deployment	Cloudflare Workers and Wrangler	Build, configure, migrate and deploy the application.

3.1 Single Full-Stack Application

There will be no separately deployed backend application. The same Next.js application will contain:

- Frontend pages and layouts.
- Reusable UI components.
- Feature-specific modules.
- Server-side API route handlers.
- Business services and calculation functions.

- Database access and query functions.
- Authentication and authorisation logic.
- Shared validation schemas and TypeScript types.

The frontend will call the application's own API routes. The API routes will call internal service functions, which will interact with D1 and R2.

3.2 Backend Runtime

Server-side code must be compatible with the Cloudflare Workers runtime and the selected Next.js-to-Workers deployment adapter. The deployment configuration must be tested against the actual application dependencies before production release.

The architecture does not require a separate Python server. Business logic should use runtime-compatible TypeScript and Web Platform APIs wherever practical. If a required library cannot run in the Workers environment, choose a compatible library or implement the required functionality within the same application.

4 Single Root Project Structure

All source code will live in one repository and one root project directory, similar to the structure used for the Ostaad website. There will be no top-level `frontend/` or `backend/` directories.

A suggested structure is:

```
ostaad-erp/
|
|-- public/
|   |-- assets/
|   |-- images/
|   |-- icons/
|
|-- src/
|   |-- app/
|   |   |-- (auth)/
|   |   |-- (dashboard)/
|   |   |-- projects/
|   |   |-- preconstruction/
|   |   |-- boq/
|   |   |-- procurement/
|   |   |-- finance/
|   |   |-- inventory/
|   |   |-- reports/
|   |   |-- api/
|   |       |-- auth/
|   |       |-- companies/
|   |       |-- projects/
|   |       |-- preconstruction/
|   |       |-- boq/
|   |       |-- vendors/
|   |       |-- rfqs/
|   |       |-- quotations/
|   |       |-- procurement/
|   |       |-- finance/
|   |       |-- inventory/
|   |       |-- documents/
|   |
|   |-- components/
|   |   |-- ui/
|   |   |-- layout/
|   |   |-- forms/
|   |   |-- tables/
|   |   |-- charts/
```

```
|   |
|   |-- modules/
|   |   |-- preconstruction/
|   |   |-- projects/
|   |   |-- boq/
|   |   |-- vendors/
|   |   |-- procurement/
|   |   |-- finance/
|   |   |-- inventory/
|   |   |-- reports/
|   |
|   |-- lib/
|   |   |-- db/
|   |   |-- auth/
|   |   |-- storage/
|   |   |-- permissions/
|   |   |-- validations/
|   |   |-- utils/
|   |
|   |-- hooks/
|   |-- types/
|   |-- styles/
|
|-- migrations/
|-- scripts/
|-- tests/
|-- .env.example
|-- package.json
|-- tsconfig.json
|-- next.config.ts
|-- wrangler.jsonc
|-- README.md
```

The structure is a logical organisation guide. Exact folders may be adjusted to the selected Next.js routing and Cloudflare deployment adapter. The key requirement is that the entire application remains in one root project and one deployment.

4.1 Code Organisation Rules

- Keep page and layout code inside `src/app`.
- Keep reusable visual elements inside `src/components`.
- Keep feature-specific code grouped inside `src/modules`.
- Keep database, authentication and storage helpers inside `src/lib`.
- Keep API route handlers inside `src/app/api`.
- Keep database migrations in the root `migrations` directory.
- Do not create a separate backend project or duplicate shared types.
- Keep database queries and business rules out of UI components.

5 Cloudflare Infrastructure Design

5.1 Cloudflare Worker

Only one Worker will be deployed for the ERP.

It will serve the frontend application and execute backend API routes. It will also access D1 and R2 using bindings.

Responsibilities include:

- Serve application pages and assets.
- Handle login, sessions and permissions.
- Expose API routes.

- Execute business logic and calculations.
- Read and write structured records in D1.
- Upload and retrieve files through R2.
- Generate reports and authorised document responses.

5.2 Cloudflare D1

D1 will be the primary and only relational database.

It will store:

- Company and user information.
- Projects and project members.
- Pre-construction records.
- Cost estimates and BOQs.
- Vendors and quotations.
- RFQs and purchase orders.
- Budgets and financial transactions.
- Inventory and GRN records.
- Approvals and audit logs.

D1 is SQLite-based. Database design must account for SQLite compatibility and D1 operational limits.

The system will use:

- Primary and foreign keys.
- Appropriate indexes.
- Version-controlled database migrations.
- Prepared and parameterised queries.
- Supported transaction mechanisms.
- Company-level data isolation.

5.3 Cloudflare R2

R2 will store actual file contents, including:

- Site photographs and inspection evidence.
- Project drawings and uploaded blueprints.
- Soil investigation reports.
- BOQ Excel files and exports.
- Vendor quotations.
- Purchase orders and invoices.
- Permit certificates and project attachments.

D1 will store each file's metadata, R2 object key, company ID, project ID, document type, owner and upload details. The actual file bytes will remain in R2.

5.4 Cloudflare KV

KV is optional and is not part of the core data architecture. It may be used for non-critical cached lookup data or configuration values where eventual consistency is acceptable.

KV must not be the authoritative source for financial, procurement or inventory records.

5.5 Services Not Required Initially

The initial design will not introduce:

- A second Worker for the backend.
- A separate Python application server.
- External PostgreSQL or another database provider.
- Microservices or independent module deployments.
- Cloudflare Queues or Durable Objects without a clear need.
- A separate caching layer unless measured performance requires it.

6 Functional Modules

The ERP will contain the following functional modules.

6.1 Module 1: Pre-Construction Planning

This module will incorporate all 15 tools from the Pre-Construction Planning Excel workbook.

No.	Tool	ERP Functionality
1	Site Survey Checklist	Record site conditions, accessibility, utilities, surroundings, restrictions and inspection observations. Support critical and non-critical checkpoints and GO/NO-GO decisions.
2	Cost Estimator	Estimate project costs using work heads, quantities, rates, labour, materials, soft costs, taxes and contingency.
3	BOQ Template	Create, edit and maintain item-wise BOQs with descriptions, units, quantities, rates, amounts and executed quantities. Support Excel import and export.
4	Timeline Gantt	Create project schedules, tasks, dependencies, start dates, end dates, milestones and progress tracking.
5	Permit Tracker	Track statutory approvals, application dates, fees, responsible persons, status and expiry dates.
6	Plot Area Calculator	Calculate regular and irregular plot areas using measurements, geometry-based formulas and multiple units.
7	Vendor Comparison	Compare vendors using price, quality, delivery, reliability, payment terms and weighted evaluation criteria.
8	Soil Test Tracker	Maintain borehole records, soil layers, SPT N-values, groundwater levels and safe bearing capacity information.
9	Feasibility Report	Evaluate investment, funding, EMI, estimated returns, break-even rates, sale or rental scenarios and margins.
10	Phase Planner	Divide construction into phases and track duration, planned expenditure, cumulative expenditure and progress.
11	Site Layout Planner	Plan temporary and permanent site zones, storage, office areas, access, utilities and construction facilities.
12	Zoning and Setback Calculator	Record plot restrictions, setbacks, FAR, ground coverage, height limits and permissible buildable area.
13	Cost vs Budget	Compare approved budgets, estimates, committed costs, actual expenses and remaining contingency.
14	Selection Scorecard	Evaluate project options using mandatory pass/fail gates and weighted technical, financial and practical criteria.
15	Project Kickoff Checklist	Track readiness, documentation, responsibilities, safety, approvals and mobilisation tasks.

6.1.1 Pre-Construction Workflow

1. Calculate plot area and check zoning restrictions.
2. Complete the site survey.
3. Record soil investigation results.
4. Prepare the initial cost estimate.
5. Generate the feasibility report.
6. Evaluate project options.
7. Prepare and finalise the BOQ.
8. Compare and shortlist vendors.
9. Prepare project timelines and phase plans.

10. Complete permit tracking and site layout planning.
11. Finalise the project budget.
12. Complete the kickoff checklist before execution.

This is a recommended sequence, not a rigid restriction. Activities may happen in parallel depending on project needs.

6.2 Module 2: Project Management

- Create and configure projects.
- Assign project managers and team members.
- Maintain client and site information.
- Upload project documents.
- Track project status and milestones.
- Link pre-construction records to projects.
- Maintain project activity history.
- Monitor overall project progress.

6.3 Module 3: BOQ and Cost Management

- Create BOQ sections and item descriptions.
- Define units, quantities, rates and amounts.
- Import BOQs from Excel.
- Edit and revise BOQ items.
- Maintain BOQ version history.
- Map BOQ items to construction materials.
- Generate material requirements.
- Track planned, ordered, received and executed quantities.
- Compare estimated and actual costs.
- Maintain Running Account (RA) bill records.

Blueprint-to-BOQ exclusion:
Automatic blueprint-to-BOQ conversion is explicitly outside the scope of this development phase. The ERP will accept manually prepared BOQs and imported BOQ files. Blueprint files may be uploaded and stored in R2, but automatic drawing interpretation, CAD quantity extraction and AI-based BOQ generation will not be implemented now.

The BOQ data model and APIs should allow a future blueprint-to-BOQ engine to integrate without requiring a redesign of procurement and finance modules.

6.4 Module 4: Vendor Management and Procurement

- Maintain a central vendor directory.
- Store vendor contact and business information.
- Maintain vendor categories and material specialisations.
- Create Requests for Quotation (RFQs).
- Send material requirements to selected vendors.
- Record vendor quotations.
- Compare prices, taxes, delivery and payment terms.
- Generate comparative statements.
- Rank vendors using selection criteria.
- Recommend vendors for approval.
- Generate Purchase Orders (POs).
- Track PO status, delivery and outstanding quantities.

Vendor master information will be logically separated from company-specific quotations, negotiated prices and commercial records.

6.5 Module 5: Finance and Accounts

- Project-wise budget allocation.
- Cost estimation and budget approval.
- Vendor quotation review.
- Purchase approval and rejection.
- Applicable tax and GST calculations.
- Purchase and expense records.
- Vendor payable tracking.
- Client receivable tracking.
- Advance payments and payment status.
- Invoice and bill management.
- Budget versus actual expenditure reports.
- Financial audit history.

Financial approval must be recorded before an authorised purchase order is released.

6.6 Module 6: Site Operations and Inventory

- Maintain site-wise material inventory.
- Record Goods Receipt Notes (GRNs).
- Verify received materials against purchase orders.
- Record accepted, rejected and damaged quantities.
- Track material inward and outward movements.
- Record material consumption.
- Track stock transfers between sites.
- Maintain stock balances and reorder information.
- Record site expenses and daily activities.
- Maintain material issue and return records.

Inventory balances will be derived from stock transactions rather than relying exclusively on manually edited quantities.

6.7 Module 7: Reports and Administration

- Project summary and progress reports.
- Pre-construction readiness reports.
- BOQ and cost reports.
- Vendor comparison reports.
- Procurement and PO reports.
- Inventory and GRN reports.
- Finance and budget reports.
- User, role and permission management.
- System configuration and master data.
- Audit logs and activity history.

7 User Roles and Dashboards

The ERP will use Role-Based Access Control (RBAC). Users will see dashboards and actions according to their assigned roles and company/project permissions.

Role	Responsibilities
Super Admin	Manage platform configuration, companies, system-level settings and access.
Company Admin	Manage company users, roles, projects, vendors, master data and company-wide settings.

Project Manager	Create projects, complete pre-construction planning, manage BOQs, schedules, budgets and project progress.
Procurement Manager	Manage RFQs, vendor quotations, comparisons, recommendations and purchase orders.
Finance Approver	Review budgets, quotations, taxes, financial exposure and approve or reject purchases.
Site Engineer	View project plans, update site progress, report quantities and request materials.
Store Manager	Record GRNs, stock movements, material issues and inventory.
Auditor / Viewer	View permitted records without modifying them.

A user may have multiple roles, subject to company permissions and approval policies.

8 End-to-End Business Workflow

The primary workflow will connect planning, BOQ, procurement, finance and site operations.



Figure 2: Primary project-to-procurement workflow

Every stage must maintain its own status, responsible user, timestamps and supporting documents.

9 Database Architecture

Cloudflare D1 will act as the central source of truth for structured ERP information. It will use a relational design with explicit relationships between companies, projects, BOQs, vendors, quotations, purchase orders and inventory.

9.1 Main Database Entities

Entity Group	Main Tables
Organisation	companies, company_settings, users, roles, permissions, user_roles, company_memberships
Projects	projects, project_members, project_sites, project_documents, project_activities
Pre-Construction	site_surveys, survey_checkpoints, plot_measurements, soil_tests, zoning_records, feasibility_reports, permits, site_layouts, kick-off_tasks
Planning	cost_estimates, cost_estimate_items, project_timelines, time-line_tasks, project_phases, selection_scorecards
BOQ	boqs, boq_versions, boq_sections, boq_items, boq_revisions
Vendors	vendors, vendor_categories, vendor_contacts, ven-dor_evaluations
Procurement	material_requirements, rfqs, rfq_items, quotations, quota-tion_items, comparative_statements, purchase_orders, pur-chase_order_items
Finance	budgets, budget_items, expenses, invoices, payments, ap-provals, financial_transactions
Inventory	warehouses, inventory_items, stock_ledger, goods_receipts, grn_items, material_issues, stock_transfers
System	audit_logs, notifications, attachments, activity_history

9.2 Database Design Rules

- Use primary keys and foreign keys.
- Associate company-owned records with company IDs.
- Associate project-specific records with project IDs.
- Use indexes for frequently queried fields.
- Use parameterised SQL queries.
- Maintain version-controlled schema migrations.
- Store money as integer minor units (for example, paise) or a consistently defined fixed-precision representation.
- Avoid PostgreSQL-specific functions and data types.
- Store file contents in R2 and metadata in D1.
- Preserve approved BOQ, budget and purchase revisions.
- Maintain created, updated and responsible-user fields.

9.3 Multi-Company Architecture

The application will support multiple companies through one shared application and one shared D1 database. Every company-owned record must contain a company identifier.

The backend will determine the active company using the authenticated user's authorised company membership. All database queries must enforce company-level isolation. Changing an identifier in a frontend request must never allow a user to access another company's records.

The central vendor directory may contain shared vendor master information. Company-specific quotations, negotiated rates, purchase orders and financial records must remain isolated.

10 API Architecture

The backend API routes will be part of the same Next.js application and the same Worker deployment. They will be organised by business domain.

Example route structure:

- `/api/auth/` — Authentication and sessions.
- `/api/companies/` — Company management.
- `/api/users/` — Users and permissions.
- `/api/projects/` — Project management.
- `/api/preconstruction/` — Pre-construction tools.
- `/api/boq/` — BOQ management.
- `/api/costs/` — Estimates and budgets.
- `/api/vendors/` — Vendor directory.
- `/api/rfqs/` — Requests for quotations.
- `/api/quotations/` — Vendor quotations.
- `/api/procurement/` — Procurement workflows.
- `/api/purchase-orders/` — Purchase orders.
- `/api/finance/` — Finance and approvals.
- `/api/inventory/` — Inventory management.
- `/api/grn/` — Goods receipt.
- `/api/reports/` — Reports and exports.
- `/api/documents/` — Document metadata and access.

Each endpoint must validate the authenticated user, company membership, project access and required permissions. API route handlers should remain thin and call reusable service functions in the relevant module.

11 Documents and File Management

Cloudflare R2 will store the actual file contents.

D1 will store:

- Document ID.
- Company ID and project ID.
- Original file name.
- R2 object key.
- Document category.
- File type and size.
- Uploaded by.
- Upload timestamp.

Private documents must not be exposed through public object URLs. The application will check authorisation before returning a file or issuing a restricted access URL.

12 Security, Reliability and Audit

The ERP must implement:

- Secure login and session handling.
- Secure password hashing if passwords are stored.
- Role-based and company-level permissions.
- Server-side company and project isolation.
- Backend validation for important operations.
- Secure document access.
- Audit logs for financial and operational changes.
- Database backup and recovery procedures.
- Input validation and protection against common attacks.
- Approval records with user, date, time and decision.
- Controlled modification of approved BOQs, budgets and POs.
- Safe handling of concurrent financial and inventory operations.

Financial and inventory operations must use supported D1 transaction mechanisms and appropriate constraints. Important operations should be idempotent where retries are possible.

The application must not rely on in-memory state persisting between Worker requests or on long-running background processes completing within a normal request.

13 Deployment Architecture

The complete production deployment will remain on Cloudflare. There will be one application deployment and one Worker.

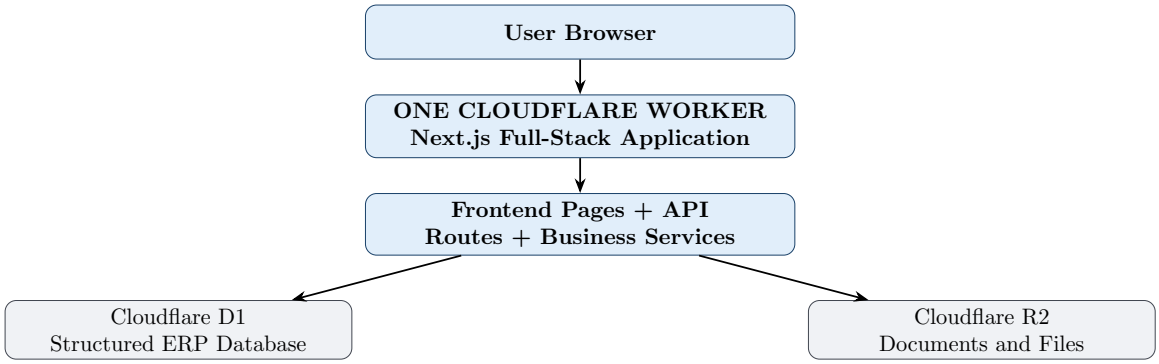


Figure 3: Single-Worker deployment on Cloudflare

13.1 Deployment Responsibilities

- **One Worker:** Serves the frontend and runs backend API routes.
- **D1:** Stores all structured ERP information.
- **R2:** Stores files and documents.
- **KV:** Optional cache only if a real requirement emerges.
- **Wrangler:** Manages Worker configuration, bindings, migrations and deployment.

The Worker will access D1 and R2 through configured Cloudflare bindings. The frontend will call same-origin API routes within the application. There will be no separate backend URL or second Worker in this design.

The exact build and deployment adapter must be selected and verified for the Next.js version and Cloudflare Workers runtime used by the project. The adapter is a deployment detail and does not change the single-application design.

14 Development Phases

Development will be prioritised around the business workflow.

Phase	Modules	Expected Outcome
Phase 1	Authentication, company management, user roles, D1 schema and project creation	Secure multi-company ERP foundation.
Phase 2	All 15 pre-construction tools	Complete project planning, feasibility and readiness management.
Phase 3	BOQ, cost estimation, budget and material requirements	Manual and imported BOQ management with project-wise cost control.
Phase 4	Vendor management, RFQs, quotations, vendor comparison and procurement	Complete vendor selection and purchasing workflow.
Phase 5	Finance, approvals, purchase orders and invoices	Controlled financial operations and purchasing.
Phase 6	GRN, inventory, site operations and reports	End-to-end procurement-to-site material tracking.

Phase 7	Testing, security, audit, backups and deployment	Pilot-ready and production-ready ERP foundation.

15 Features Outside the Current Scope

The following functionality will not be developed in this phase:

- Automatic blueprint-to-BOQ conversion.
- AI-based drawing interpretation.
- Automatic CAD quantity extraction.
- Advanced BIM and IFC model processing.
- Microservices architecture.
- Separate frontend and backend deployments.
- Independent databases for each module.
- Advanced AI-based vendor selection.

These features can be evaluated separately in future phases. The core BOQ, project, procurement and finance data models will remain suitable for future integration.

16 Final Architecture Summary

Ostaad ERP will be a Cloudflare-native, modular monolithic, multi-company construction management application.

The complete architecture will consist of:

1. One Next.js full-stack application in a single root project.
2. One Cloudflare Worker deployment for frontend and backend.
3. Cloudflare D1 as the only relational database.
4. Cloudflare R2 for documents and file storage.
5. Optional Cloudflare KV for suitable non-critical caching.
6. Same-origin API routes within the full-stack application.
7. Company-level data isolation and role-based access.
8. Central audit, approval and transaction tracking.

The ERP will manage the business workflow from pre-construction planning to procurement, finance and site inventory management. Automatic blueprint-to-BOQ conversion is excluded from the current development phase.

The emphasis will remain on business functionality, accurate records, reliable workflows, ease of use and a lightweight technical architecture.

OSTAAD ERP

One integrated construction management system.

One Cloudflare Worker + D1 + R2